

Final Project Report

CSE 3241

Om Kurkure
Mitchell Zaher
Sayan Bisi
Cole Taglione

Table of Contents

Content	Page Number
Introduction	2
SECTION I: Entity-Relationship Model	2
Relational Schema	3
Views implemented for Database	6
Indexes	7
SECTION II: Relation Descriptions	8
Sample Queries	11
Insert and Delete Syntax	17
Section III: Graded Checkpoint Documents	21

Introduction

This is the final project report for CSE 3241 database project. The objective of this project was to design an information management system and database for the Bits & Books store to help manage sales and inventory operations. This document provides an overview of the steps taken by the team to model and create the database. Along with this, there are some sample SQL queries and views provided that were used to test the database.

SECTION I: Database Description

Entity-Relationship Model

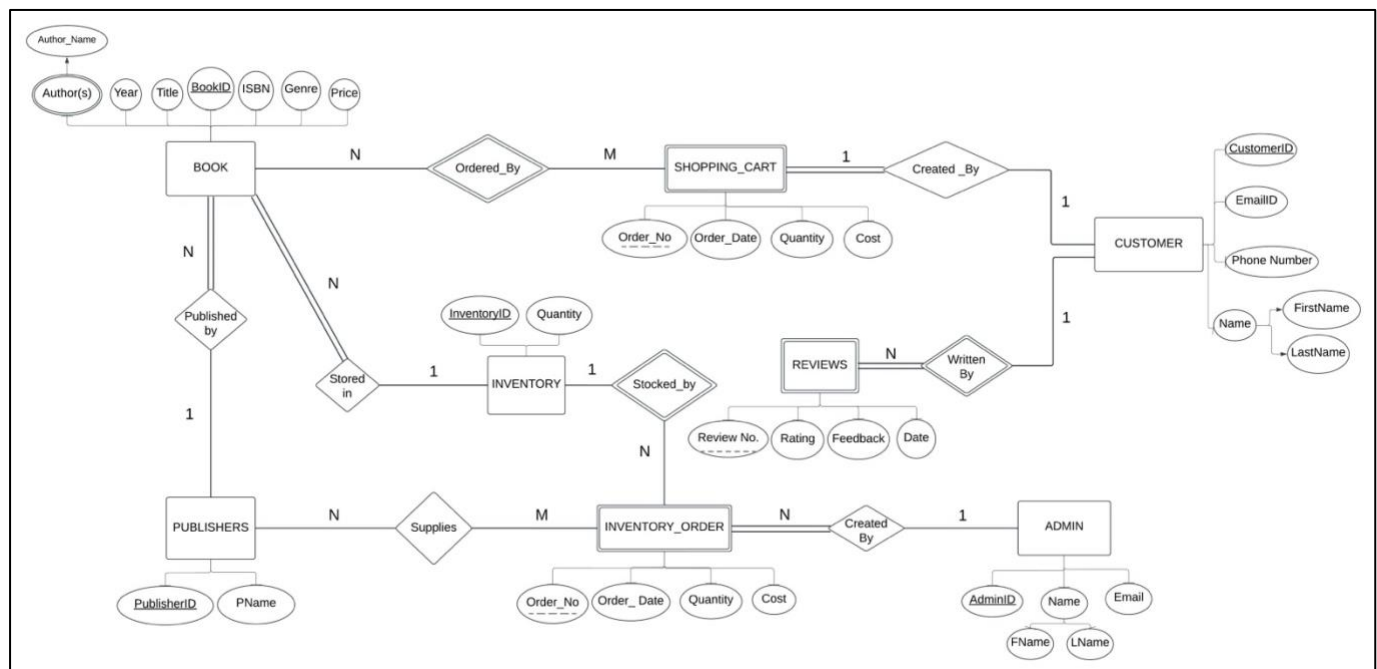


Figure 1. Entity-Relationship model for the Bookstore database

Figure 1. shows the final Entity-Relationship model that the team designed for the project. The model for the database mainly focuses on the real-world entities that a bookstore uses. It records the books that bookstore keeps in the BOOK entity. The PUBLISHERS and INVENTORY entities are used to keep track of the publishers associated with the bookstore and quantities of the books in the bookstore. The CUSTOMER and SHOPPING_CART entities store information about user interactions with the bookstore and customer details. Another entity associated with the customer is the REVIEW entity that keeps track of reviews posted by customers about the bookstore. The ADMIN entity model is responsible for managing the inventory by creating stocking orders, whose information is stored INVENTORY_ORDER entity.

Relational Schema

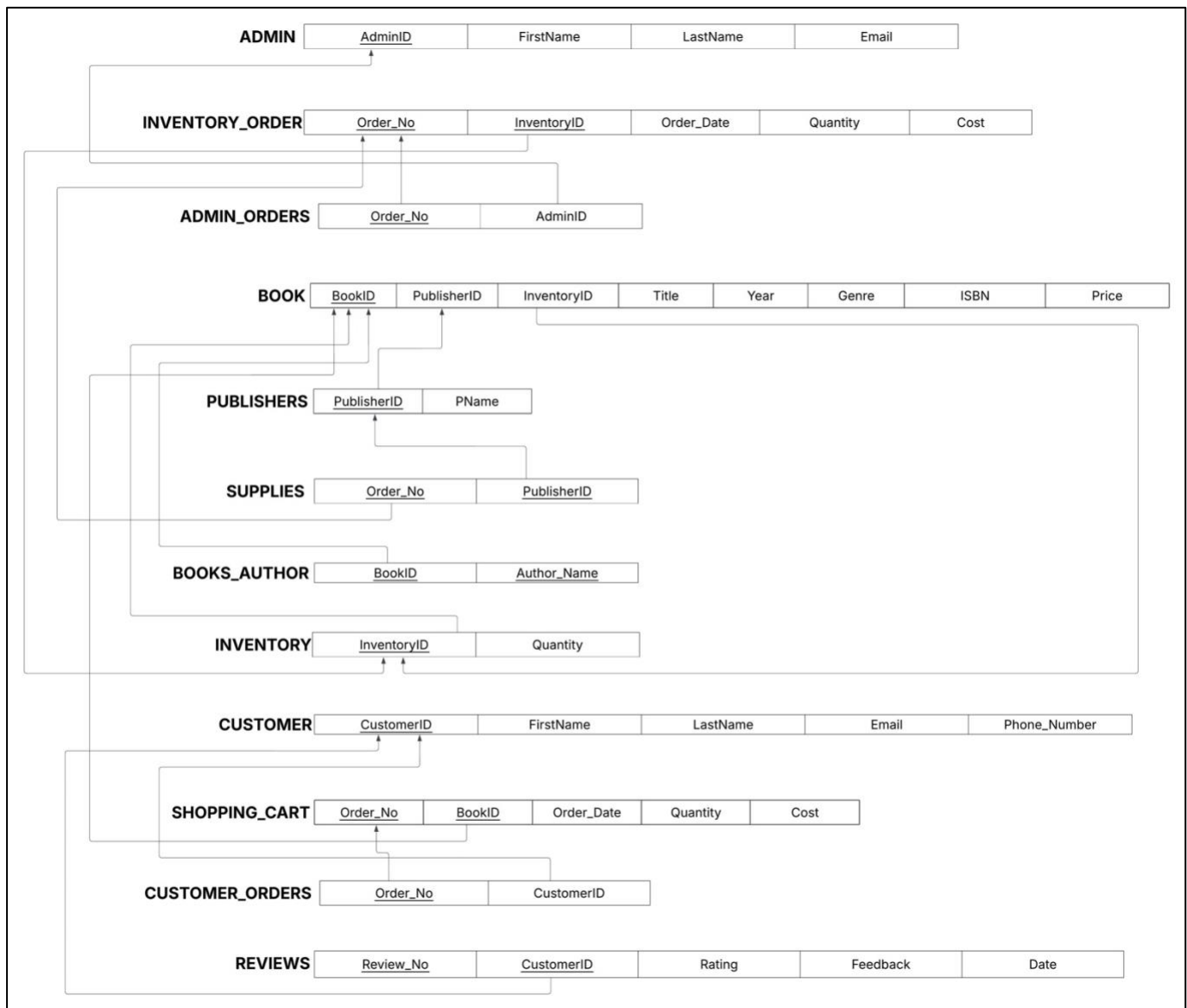


Figure 2. Relational Schema for database

Figure 2. shows the final relational schema for the database constructed from the Entity-Relationship model shown Figure 1. Here is a breakdown of all relations (FK denotes foreign key):

1. **ADMIN Relation:**

ADMIN(AdminID#, FirstName, LastName, Email)

AdminID is the primary key. Email is a candidate key. This relation is in BCNF.

Functional Dependencies:

{AdminID} -> {FirstName, LastName, Email}

{Email} -> {AdminID, FirstName, LastName}

2. **INVENTORY_ORDER Relation:**

INVENTORY_ORDER(Order_No#, Order_Date, InventoryID^(FK), Quantity, Cost)

The Primary key is the composite key {Order_No#, InventoryID#}. This relation is in BCNF.

Functional Dependencies:

{Order_No, InventoryID} -> {Order_Date, Quantity, Cost}

3. **ADMIN_ORDERS Relation:**

ADMIN_ORDERS(Order_No#^(FK), AdminID^(FK))

The Primary key is Order_No#. This relation is in BCNF.

Functional Dependencies:

{Order_No} -> {AdminID}

4. **BOOK Relation:**

BOOK(BookID#, PublisherID^(FK), InventoryID^(FK), Title, Year, Genre, ISBN, Price)

The Primary key is BookID#. InventoryID and ISBN are candidate keys. This relation is in BCNF.

Functional Dependencies:

{BookID} -> {PublisherID, InventoryID, Title, Year, Genre, ISBN, Price}

{InventoryID} -> {BookID, PublisherID, Title, Year, Genre, ISBN, Price}

{ISBN} -> {BookID, InventoryID, PublisherID, Title, Year, Genre, Price}

5. **PUBLISHERS Relation:**

PUBLISHERS(PublisherID#, PName)

The Primary key is PublisherID. This relation is in BCNF.

Functional Dependencies:

{PublisherID} -> {PName}

6. **SUPPLIES Relation:**

SUPPLIES(Order_No^(FK), PublisherID^(FK))

The primary key is the composite key {Order_No, PublisherID}. There are no meaningful functional dependencies for this relation because there two attributes in the relation, which form the primary key for the relation. Therefore, by default the relation is in BCNF.

7. **BOOKS_AUTHOR Relation:**

BOOKS_AUTHOR(BookID#^(FK), Author_Name)

The primary is the composity key {BookID, Author_Name}. There are no meaningful functional dependencies for this relation because there are two attributes in the relation, which form the primary key for the relation. Therefore, by default the relation is in BCNF.

8. **INVENTORY Relation**

INVENTORY(InventoryID#, Quantity)

The primary key is the InventoryID attribute. This relation is in BCNF.

Functional Dependencies:

{InventoryID} -> {Quantity}

9. **CUSTOMER Relation:**

CUSTOMER(CustomerID#, FirstName, LastName, Email, Phone_Number)

The primary key is CustomerID. Email and Phone_Number are candidate keys. This is relation is in BCNF.

Functional Dependencies:

{CustomerID} -> {FirstName, LastName, Email, Phone_Number}

{Email} -> {CustomerID, FirstName, LastName, Phone_Number}

{Phone_Number} -> {CustomerID, FirstName, LastName, Email}

10. **SHOPPING_CART Relation:**

SHOPPING_CART(Order_No#, Order_Date, BookID^(FK), Quantity, Cost)

The primary key is the composite key {Order_No, BookID}. This relation is in BCNF.

Functional Dependencies:

{Order_No, BookID} -> {Order_Date, Quantity, Cost}

11. **CUSTOMER_ORDERS Relation:**

CUSTOMER_ORDERS(Order_No^(FK), CustomerID^(FK))

The primary key is Order_No. This relation is in BCNF.

Functional Dependencies:

{Order_No} -> {CustomerID}

12. **REVIEWS Relation:**

REVIEWS(Review_No#, CustomerID^(FK), Rating, Feedback, Date)

The primary key is the composite key {Review_No, CustomerID}. This relation is in BCNF.

Functional Dependencies:

{Review_No, CustomerID} -> {Rating, Feedback, Date}

Views Implemented for Database

View 1: Customer Orders - This view records the total amount spent by each customer in the bookstore. Provides information about frequent customers to the bookstore.

```
CREATE VIEW Customer_Orders_Total AS
SELECT Customer.CustomerID, Customer.FirstName, Customer.LastName, Customer.Email,
COUNT(Shopping_Cart.Order_No) AS Total_Orders , SUM(Shopping_Cart.Cost) AS TotalSpent
FROM Customer_Orders
JOIN Shopping_Cart ON Customer_Orders.Order_No = Shopping_Cart.Order_No
JOIN Customer ON Customer.CustomerID = Customer_Orders.CustomerID
GROUP BY Customer_Orders.CustomerID;
```

Relational Algebra:

ORDERS $\leftarrow$ **CUSTOMER_ORDERS** * **SHOPPING_CART**

CUSTOMER_DETAILS $\leftarrow$ **ORDERS** $\bowtie_{\text{ORDERS.CustomerID} = \text{Customer.CustomerID}}$ (**CUSTOMER**)

$\rho_{\text{AGGREGATE_DETAILS}}(\text{CustomerID}, \text{Total_Orders}, \text{TotalSpent}) (\text{CustomerID} \bowtie \text{COUNT Order_No SUM Cost } (\text{CUSTOMER_DETAILS}))$

$\pi_{\text{CustomerID, FirstName, LastName, Email, Total_Orders, Total_Spent}} (\text{CUSTOMER_DETAILS} * \text{AGGREGATE_DETAILS})$

CustomerID	FirstName	LastName	Email	Total_Orders	TotalSpent
1	Michael	Scott	michael.scott@email.com	3	139.87
2	Pam	Beesly	pam.beesly@email.com	1	2195
3	Jim	Halpert	jim.halpert@email.com	1	7.99
4	Dwight	Schrute	dwight.schrute@email.com	1	41.99
5	Andy	Bernard	andy.bernard@email.com	4	136.64
6	Phyllis	Vance	phyllis.vance@email.com	3	273.9
7	Stanley	Hudson	stanley.hudson@email.com	1	26
8	Kevin	Malone	kevin.malone@email.com	2	48.870000000000005
9	Oscar	Martinez	oscar.martinez@email.com	1	14.25
10	Angela	Martin	angela.martin@email.com	1	245
11	Toby	Flenderson	toby.flenderson@email.com	1	15.98
12	Creed	Bratton	creed.bratton@email.com	2	42.5
13	Meredith	Palmer	meredith.palmer@email.com	1	83
14	Kelly	Kapoor	kelly.kapoor@email.com	1	120
15	Ryan	Howard	ryan.howard@email.com	1	29.99
16	Darryl	Philbin	darryl.philbin@email.com	1	49.9
17	Roy	Anderson	roy.anderson@email.com	3	114.99000000000001
18	Jan	Levinson	jan.levinson@email.com	2	171.95
19	David	Wallace	david.wallace@email.com	1	79.9
20	Holly	Flax	holly.flax@email.com	1	199.75

Figure 3. Customer Orders view example output

View 2: Customer Reviews - This view displays the average rating per customer and total number of ratings they have posted. Includes customer information. This provides information about customer satisfaction and a contact point for customers that have not had a good experience.

```
CREATE VIEW Customer_Reviews AS
SELECT Customer.FirstName, Customer.LastName, Customer.Email,
COUNT(Reviews.Review_No) as Review_Count, AVG(Reviews.Rating) as Avg_Rating
FROM Customer
JOIN Reviews ON Customer.CustomerID = Reviews.CustomerID
GROUP BY Customer.CustomerID;
```

Relational Algebra:

CUSTOMER_REVIEWS $\leftarrow$ CUSTOMER * REVIEWS

ρ AGGREGATE_DETAILS(CustomerID, Review_Count, Avg_Rating) (CustomerID $\bowtie$ COUNT Review_No AVG Rating(CUSTOMER_REVIEWS))

π FirstName, LastName, Email, Review_Count, Avg_Rating (CUSTOMER_REVIEW * AGGREGATE_DETAILS)

FirstName	LastName	Email	Review_Count	Avg_Rating
Michael	Scott	michael.scott@email.com	2	2
Pam	Beesly	pam.beesly@email.com	1	4
Jim	Halpert	jim.halpert@email.com	1	1
Dwight	Schrute	dwight.schrute@email.com	3	4.33333333333333
Andy	Bernard	andy.bernard@email.com	1	1
Phyllis	Vance	phyllis.vance@email.com	1	5
Stanley	Hudson	stanley.hudson@email.com	1	4
Kevin	Malone	kevin.malone@email.com	1	1
Oscar	Martinez	oscar.martinez@email.com	1	5
Angela	Martin	angela.martin@email.com	1	5
Toby	Flenderson	toby.flenderson@email.com	1	2
Creed	Bratton	creed.bratton@email.com	1	5
Kelly	Kapoor	kelly.kapoor@email.com	3	2.66666666666667
Darryl	Philbin	darryl.philbin@email.com	1	4
David	Wallace	david.wallace@email.com	2	3.5

Figure 4. Customer Reviews view example output

Indexes

- CUSTOMER_ORDERS.(CustomerID):

Since the CUSTOMER_ORDERS table contains information about all orders made by the customer, it can be accessed quite frequently. Therefore, there is an index on the CustomerID column of the CUSTOMER_ORDERS relation.

- BOOK.(BookID):

The BOOK table contains information (Title, Year of Publication, ISBN, etc.) about every book that is either stored in the inventory or ordered/purchased from the bookstore. The primary key of the BOOK table is BookID, which is also referred to by other tables (BOOKS_AUTHOR and SHOPPING_CART) through foreign keys, hence showing the frequency of this attribute being accessed in the database. Thus, it makes sense to use this as an index.

For both of these indexes although large amount of maintenance would be required for the indexes since the CUSTOMER_ORDERS and BOOK relations could be updated frequently, that cost can be tolerated since these relations are the most frequently accessed relations in the database.

SECTION II: User Manual

Relation Descriptions

1. ADMIN Relation:

ADMIN(AdminID#, FirstName, LastName, Email)

This relation keeps track of the admins associated with the bookstore. Admins are responsible in keeping track of book quantity and placing stocking orders.

Attributes:

- AdminID – INT data type and has a NOT NULL constraint. ID to uniquely identify Admin.
- FirstName – VARCHAR data type and has a NOT NULL constraint. Stores the first name of the Admin.
- LastName – VARCHAR data type and has a NOT NULL constraint. Stores the last name of the Admin.
- Email – VARCHAR data type and has a NOT NULL constraint. Stores the email id for the admin which serves as a point of contact in the real-world.

2. INVENTORY_ORDER Relation:

INVENTORY_ORDER(Order_No#, Order_Date, InventoryID^(FK), Quantity, Cost)

This relation keeps track of the stock orders made by Admins.

Attributes:

- Order_No – INT data type and has a NOT NULL constraint. Keeps track of total orders made.
- InventoryID – INT data type and has a NOT NULL constraint. Holds the inventory number that the order is made for.
- Order_Date – DATE data type and has a NOT NULL constraint. Stores the date of the order.
- Quantity – INT data type and has a NOT NULL constraint. Stores quantity of the book ordered.
- Cost – REAL data type and has a NOT NULL constraint. Stores cost of the total order, not the individual book price.

3. ADMIN_ORDERS Relation:

ADMIN_ORDERS(Order_No#^(FK), AdminID^(FK))

This relation keeps track of admin associated with each stocking order for the bookstore.

Attributes:

- Order_No – INT data type and has a NOT NULL constraint. References an order from the INVENTORY_ORDER relation.
- AdminID – INT data type and has a NOT NULL constraint. Stores the admin id who made a particular inventory order

4. BOOK Relation:

BOOK(BookID#, PublisherID^(FK), InventoryID^(FK), Title, Year, Genre, ISBN, Price)

This relation keeps track of the books that the bookstore has available to customers.

Attributes:

- BookID – INT data type and has a NOT NULL constraint. Identifies each book by a unique ID number.
- PublisherID – INT data type and has a NOT NULL constraint. Stores the publisher id for the book.
- InventoryID – INT data type and has a NOT NULL constraint. Stores the inventory in which the book is stored.
- Title – VARCHAR data type and has a NOT NULL constraint. Stores the title of the book.
- Year – INT data type. Stores the year the book was published.
- Genre – VARCHAR data type. Stores the genre of the book.
- ISBN – VARCHAR data type and has NOT NULL and UNIQUE constraint. Stores the ISBN number for the book.
- Price – REAL data type and has a NOT NULL constraint. Stores the price of the book.

5. **PUBLISHERS Relation:**

PUBLISHERS(PublisherID#, PName)

This relation keeps track of the publishers associated with the bookstore.

Attributes:

- PublisherID – INT data type and has a NOT NULL constraint. Unique ID number for each publisher associated with the bookstore.
- PName – VARCHAR data type and has a NOT NULL constraint. Stores the name of the publisher.

6. **SUPPLIES Relation:**

SUPPLIES(Order_No^(FK), PublisherID^(FK))

This relation keeps track of the stocking orders and the publishers that supplied the books for them.

Attributes:

- Order_No – INT data type and has a NOT NULL constraint. References a order no from the Inventory_Order relation.
- PublisherID – INT data type and has a NOT NULL constraint. Stores the publisher that supplied books for inventory order.

7. **BOOKS_AUTHOR Relation:**

BOOKS_AUTHOR(BookID#^(FK), Author_Name)

This relation keeps track of books in the bookstore and their respective authors.

Attributes:

- BookID – INT data type and has a NOT NULL constraint. References a book from the book relation.
- AuthorName – VARCHAR data type and has a NOT NULL constraint. Stores the author name for a particular book id.

8. **INVENTORY Relation**

INVENTORY(InventoryID#, Quantity)

This relation keeps track of the quantity of a particular in stock.

Attributes:

- InventoryID – INT data type and has a NOT NULL constraint. Stores a unique number to identify the inventory for a particular book.
- Quantity – INT data type. Stores the quantity of a particular book.

9. **CUSTOMER Relation:**

CUSTOMER(CustomerID#, FirstName, LastName, Email, Phone_Number)

This relation keeps track of customers associated with the database.

Attributes:

- CustomerID – INT data type and has a NOT NULL constraint. Uniquely identifies a customer.
- FirstName – VARCHAR data type and has a NOT NULL constraint. Stores the first name of the customer.
- LastName – VARCHAR data type and has a NOT NULL constraint. Stores the last name of the customer.
- Email – VARCHAR data type and has a NOT NULL constraint. Stores the email id of the customer.
- Phone_Number – CHAR data type. Stores the phone number of the customer.

10. **SHOPPING_CART Relation:**

SHOPPING_CART(Order_No#, Order_Date, BookID^(FK), Quantity, Cost)

This relation keeps track of order detail for orders made by customers. This relation does hold information about the customer itself.

Attributes:

- Order_No – INT data type and has a NOT NULL constraint. Assigns a order number to a order made by a customer.
- BookID – INT data type and has a NOT NULL constraint. Stores the book id that the customer has placed an order for.
- Order_Date – DATE data type and has a NOT NULL constraint. Stores the order date for the order made by the customer.
- Quantity – INT data type and has a NOT NULL constraint. Stores the quantity for a particular book that customer has ordered.
- Cost – REAL data type and has a NOT NULL constraint. Stores the cost of the total order, not the individual book.

11. **CUSTOMER_ORDERS Relation:**

CUSTOMER_ORDERS(Order_No^(FK), CustomerID^(FK))

This relation keeps track of the CustomerIDs that made orders.

Attributes:

- Order_No – INT data type and has a NOT NULL constraint. Stores the order number for a particular made by a customer
- CustomerID – INT data type and has a NOT NULL constraint. Stores the customer id that has made the order.

12. REVIEWS Relation:

REVIEWS(Review_No#, CustomerID^(FK), Rating, Feedback, Date)

This relation stores information about reviews made by customers.

Attributes:

- Review_No – INT data type and has a NOT NULL constraint. Stores the review number for a particular review made by a customer
- CustomerID – INT data type and has a NOT NULL constraint. Stores the customer id that published the review.
- Rating – INT data type. Stores the rating published by a customer for the bookstore.
- Feedback – VARCHAR data type. Stores the feedback provided by a customer for the bookstore.
- Date – DATE data type and has a NOT NULL constraint. Stores the date that the review was published.

Sample Queries

1. Find the titles of all books by Terry Pratchett that cost less than \$10.

Relational Algebra:

PRATCHETT_ID $\leftarrow \pi_{\text{BookID}} (\sigma_{\text{Author_Name} = \text{'Terry Pratchett'}} (\text{BOOKS_AUTHOR}))$

BOOKS $\leftarrow \text{PRATCHETT_ID} \bowtie_{\text{PRATCHETT_ID.BookID} = \text{BOOK.BookID}} (\text{BOOK})$

RESULT $\leftarrow \pi_{\text{Title}} (\sigma_{\text{Price} < 10} (\text{BOOKS}))$

SQL Query:

```
SELECT BookID, Book.Title
FROM Book NATURAL JOIN Books_Author
WHERE Books_Author.AuthorName LIKE '%Pratchett%' AND Book.Price < 10;
```

2. Give all the titles and their dates of purchase made by a single customer (Assumption CustomerID = 5)

Relational Algebra:

ORDERS $\leftarrow \text{SHOPPING_CART} \bowtie_{\text{SHOPPING_CART.CustomerID} = \text{CUSTOMER_ORDERS.CustomerID}} (\text{CUSTOMER_ORDERS})$

BOOK_DETAILS $\leftarrow \text{BOOK} \bowtie_{\text{BOOK.BookID} = \text{ORDERS.BookID}} (\text{ORDERS})$

RESULT $\leftarrow \pi_{\text{Title, Order_Date}} (\sigma_{\text{CustomerID} = 5} (\text{BOOK_DETAILS}))$

SQL Query:

```
SELECT Book.Title, Shopping_Cart.Order_Date
FROM Shopping_Cart NATURAL JOIN Customer_Orders
JOIN Book ON Book.BookID = Shopping_Cart.BookID
WHERE Customer_Orders.CustomerID = 5;
```

3. Find the titles and ISBNs for all books with less than 5 copies in stock

Relational Algebra:

LOW_BOOK_IDS $\leftarrow \pi_{\text{BookID}} (\sigma_{\text{Quantity} < 5} (\text{INVENTORY}))$
BOOKS $\leftarrow \text{LOW_BOOK_IDS} \bowtie_{\text{LOW_BOOK_IDS.BookID} = \text{BOOK.BookID}} (\text{BOOK})$
RESULT $\leftarrow \pi_{\text{Title, ISBN}} (\text{BOOKS})$

SQL Query:

```
SELECT Book.Title, Book.ISBN
FROM Book JOIN Inventory ON Book.InventoryID = Inventory.InventoryID
WHERE Inventory.Quantity < 5;
```

4. Give all the customers who purchased a book by Terry Pratchett and titles of Pratchett books they purchased

Relational Algebra:

ORDERS $\leftarrow \text{SHOPPING_CART} * \text{CUSTOMER_ORDERS}$
CUSTOMER_DETAILS $\leftarrow \text{ORDERS} \bowtie_{\text{ORDERS.CustomerID} = \text{CUSTOMER.CustomerID}} (\text{CUSTOMER})$
PRATCHETT_BOOK_IDS $\leftarrow \pi_{\text{BookID}} (\sigma_{\text{Author_Name} = \text{'Terry Pratchett'}} (\text{BOOKS_AUTHOR}))$
PRATCHETT_CUST $\leftarrow \text{PRATCHETT_BOOK_IDS} \bowtie_{\text{PRATCHETT_BOOK_IDS.BookID} = \text{CUSTOMER_DETAILS.BookID}} (\text{CUSTOMER_DETAILS})$
RESULT $\leftarrow \pi_{\text{CustomerID, FirstName, LastName, Title}} (\text{PRATCHETT_CUST})$

SQL Query:

```
SELECT Customer.CustomerID, Customer.FirstName, Customer.LastName, Book.Title
FROM Shopping_Cart NATURAL JOIN Customer_Orders
JOIN Customer ON Customer_Orders.CustomerID = Customer.CustomerID
JOIN Book ON Shopping_Cart.BookID = Book.BookID
JOIN Books_Author ON Book.BookID = Books_Author.BookID
WHERE Books_Author.AuthorName LIKE '%Pratchett%';
```

5. Find the total number of books purchased by a single customer (Assume CustomerID = 1)

Relational Algebra:

CUSTOMER_BOOK_COUNT $\leftarrow \text{CustomerID} \wr_{\text{COUNT BookID}} (\text{SHOPPING_CART} * \text{CUSTOMER_ORDERS})$
RESULT $\leftarrow \sigma_{\text{CustomerID} = 1} (\text{CUSTOMERS_BOOK_COUNT})$

SQL Query:

```
SELECT CustomerID, Book_Count
FROM (SELECT Customer_Orders.CustomerID, COUNT(*) AS BOOK_COUNT
      FROM Shopping_Cart NATURAL JOIN Customer_Orders
      GROUP BY Customer_Orders.CustomerID)
WHERE CustomerID = 1;
```

6. Find the customer who has purchased the most books and the total number of books they have purchased. Return the CustomerID

Relational Algebra:

$\rho_{\text{BOOK_COUNT}(\text{CustomerID, BookCount})} (\text{CustomerID} \wr_{\text{COUNT BookID}} (\text{SHOPPING_CART} * \text{CUSTOMER_ORDERS}))$

$\rho_{\text{MAX_COUNT}}(\text{CustomerID}, \text{NumBooks}) (\text{CustomerID} \bowtie \text{MAX BookCount} (\text{BOOK_COUNT}))$
RESULT $\leftarrow \pi_{\text{CustomerID}}(\text{MAX_COUNT})$

SQL Query:

```
SELECT CustomerID, MAX(Book_Count) AS Max_Count
FROM (SELECT CustomerID, COUNT(*) AS Book_Count
      FROM Shopping_Cart NATURAL JOIN Customer_Orders
      GROUP BY CustomerID);
```

- Find count of ratings per Customer and average rating per Customer along with Customer name and email

Relational Algebra:

$\rho_{\text{REVIEW_PER_CUST}}(\text{CustomerID}, \text{NumReview}, \text{AvgRating}) (\text{CustomerID} \bowtie \text{COUNT ReviewNo}, \text{AVERAGE Rating} (\text{REVIEWS}))$
CUSTOMER_DETAILS $\leftarrow \text{REVIEW_PER_CUST} \bowtie_{\text{REVIEW_PER_CUST.CustomerID} = \text{CUSTOMER.CustomerID}} (\text{CUSTOMER})$
RESULT $\leftarrow \pi_{\text{CustomerID}, \text{FirstName}, \text{LastName}, \text{NumReview}, \text{AvgRating}} (\text{CUSTOMER_DETAILS})$

SQL Query:

```
SELECT Customer.FirstName, Customer.LastName, COUNT(Rating) AS Rating_Count,
AVG(Rating) as Avg_Rating, Customer.Email
FROM Reviews JOIN Customer ON Reviews.CustomerID = Customer.CustomerID
GROUP BY Reviews.CustomerID;
```

- Find all stock orders made for books by 'Terry Pratchett' and retrieve admin details, book title, date of order, inventory details, quantity, and cost

Relational Algebra:

ORDERS $\leftarrow \text{INVENTORY_ORDER} * \text{ADMIN_ORDERS}$
BOOK_DETAILS $\leftarrow \text{ORDERS} \bowtie_{\text{ORDERS.InventoryID} = \text{BOOK.InventoryID}} (\text{BOOK})$
ADMIN_INFO $\leftarrow \text{BOOK_DETAILS} \bowtie_{\text{BOOK_DETAILS.AdminID} = \text{ADMIN.AdminID}} (\text{ADMIN})$
PRATCHETT_BOOK_IDS $\leftarrow \pi_{\text{BookID}} (\sigma_{\text{Author_Name} = \text{'Terry Pratchett'}} (\text{BOOKS_AUTHOR}))$
PRATCHETT_ORDERS $\leftarrow \text{PRATCHETT_BOOK_IDS} \bowtie_{\text{PRATCHETT_BOOK_IDS.BookID} = \text{ADMIN_INFO.BookID}} (\text{ADMIN_INFO})$
ADMIN_DETAILS $\leftarrow \text{PRATCHETT_ORDERS} \bowtie_{\text{PRATCHETT_ORDERS.AdminID} = \text{ADMIN.AdminID}} (\text{ADMIN})$
RESULT $\leftarrow \pi_{\text{FName}, \text{LName}, \text{Title}, \text{InventoryID}, \text{Order_Date}, \text{Quantity}, \text{Cost}} (\text{PRATCHETT_ORDERS})$

SQL Query:

```
SELECT Admin.FirstName, Admin.LastName, Book.Title, Inventory_Order.InventoryID,
Inventory_Order.Order_Date, Inventory_Order.Quantity, Inventory_Order.Cost
FROM Inventory_Order NATURAL JOIN Admin_Orders
JOIN Book ON Inventory_Order.InventoryID = Book.InventoryID
JOIN ADMIN ON Admin_Orders.AdminID = ADMIN.AdminID
JOIN Books_Author ON Book.BookID = Books_Author.BookID
WHERE Books_Author.AuthorName LIKE '%Pratchett%';
```

- Find all publishers for books authored by 'Terry Pratchett' along with book titles and price

Relational Algebra:

PRATCHETT_BOOK_IDS $\leftarrow \pi_{\text{BookID}} (\sigma_{\text{Author_Name} = \text{'Terry Pratchett'}} (\text{BOOKS_AUTHOR}))$

PRATCHETT_BOOKS $\leftarrow \pi_{\text{BookID, PublisherID, Title, Price}} (\text{PRATCHETT_BOOKS_IDS} \bowtie \text{PRATCHETT_BOOK_IDS.BookID} = \text{BOOK.BookID} (\text{BOOK}))$
PRATCHETT_PUB $\leftarrow \text{PRATCHETT_BOOKS} \bowtie \text{PRATCHETT_BOOKS.PublisherID} = \text{PUBLISHER_BOOKS.PublisherID} (\text{PUBLISHERS})$
RESULT $\leftarrow \pi_{\text{PublisherID, PName, Title, Price}} (\text{PRATCHETT_PUB})$

SQL Query:

```
SELECT Publishers.PublisherID, Publishers.Pname, Book.Title, Book.Price
FROM Publishers NATURAL JOIN Book
JOIN Books_Author ON Book.BookID = Books_Author.BookID
WHERE Books_Author.AuthorName LIKE '%Pratchett%';
```

10. Provide a list of customer names, along with the total dollar amount each customer has spent

Relational Algebra:

ORDERS $\leftarrow \text{SHOPPING_CART} * \text{CUSTOMER_ORDERS}$
CUSTOMER_DETAILS $\leftarrow \text{ORDERS} \bowtie_{\text{ORDERS.CustomerID} = \text{CUSTOMER.CustomerID}} (\text{CUSTOMER})$
 $\rho_{\text{AGGREGATE_DETAILS}(\text{CustomerID, Amount_Spent})} (\text{CustomerID} \bowtie \text{SUM Cost} (\text{CUSTOMER_DETAILS}))$
RESULTS $\leftarrow \pi_{\text{FirstName, LastName, Amount_Spent}} (\text{CUSTOMER_DETAILS} * \text{AGGREGATE_DETAILS})$

SQL Query:

```
SELECT Customer.FirstName, Customer.LastName, Sum(Cost) as Amount_Spent
FROM Shopping_Cart NATURAL JOIN Customer_Orders
JOIN Customer ON Customer_Orders.CustomerID = Customer.CustomerID
GROUP BY Customer.CustomerID;
```

11. Provide a list of customer names and e-mail addresses for customers who have spent more than the average customer

Relational Algebra:

ORDERS $\leftarrow \text{SHOPPING_CART} * \text{CUSTOMER_ORDERS}$
 $\rho_{\text{AMOUNT_SPENT}(\text{CustomerID, Total_Expenditure})} (\text{CustomerID} \bowtie \text{SUM Cost} (\text{ORDERS}))$
 $\rho_{\text{AVERAGE_EXPENDITURE}(\text{Average})} (\bowtie \text{AVG Total_Expenditure} (\text{AMOUNT_SPENT}))$
CUSTOMER_DETAILS $\leftarrow \text{AMOUNT_SPENT} * \text{CUSTOMER}$
RESULTS $\leftarrow \pi_{\text{CustomerID, FirstName, LastName, Email, Total_Expenditure}} (\sigma_{\text{Total_Expenditure} > \text{AVERAGE_EXPENDITURE.Average}} (\text{CUSTOMER_DETAILS}))$

SQL Query:

```
SELECT Customer.CustomerID, Customer.FirstName, Customer.LastName, Customer.Email,
SUM(Cost) as Total_Expenditure
FROM Shopping_Cart NATURAL JOIN Customer_Orders
JOIN Customer ON Customer_Orders.CustomerID = Customer.CustomerID
GROUP BY Customer_Orders.CustomerID
HAVING Total_Expenditure >
(SELECT AVG(TotalSpent) FROM
(SELECT SUM(Cost) AS TotalSpent
FROM Shopping_Cart NATURAL JOIN Customer_Orders
GROUP BY CustomerID));
```

12. Provide a list of the titles in the database and associated total copies sold to customers, sorted from the title that has sold the most individual copies to the title that has sold the least

Relational Algebra:

ORDERS $\leftarrow$ BOOK * SHOPPING_CART

$\rho_{\text{AGGREGATE_DETAILS}(\text{BookID}, \text{Copies_Sold})}(\text{BookID} \bowtie \text{SUM Quantity}(\text{ORDERS}))$

$\pi_{\text{BookID}, \text{Title}, \text{Copies_Sold}}(\text{ORDERS} * \text{AGGREGATE_DETAILS})$

SQL Query:

```
SELECT BookID, Book.Title, SUM(Quantity) as Copies_Sold
FROM Book NATURAL JOIN Shopping_Cart
GROUP BY BookID
ORDER BY Copies_Sold DESC;
```

13. Provide a list of the titles in the database and associated dollar totals for copies sold to customers, sorted from the title that has sold the highest dollar amount to the title that has sold the smallest

Relational Algebra:

ORDERS $\leftarrow$ BOOK * SHOPPING_CART

$\rho_{\text{AGGREGATE_DETAILS}(\text{BookID}, \text{Dollar_Total})}(\text{BookID} \bowtie \text{SUM Cost}(\text{ORDERS}))$

$\pi_{\text{BookID}, \text{Title}, \text{Dollar_Total}}(\text{ORDERS} * \text{AGGREGATE_DETAILS})$

SQL Query:

```
SELECT BookID, Book.Title, SUM(Cost) as Dollar_Total
FROM Book NATURAL JOIN Shopping_Cart
GROUP BY BookID
ORDER BY Dollar_Total DESC;
```

14. Find the most popular author in the database according to copies sold. Return author name and copies sold

Relational Algebra:

ORDERS $\leftarrow$ BOOKS_AUTHOR * SHOPPING_CART

$\rho_{\text{AGGREGATE_DETAILS}(\text{BookID}, \text{AuthorName}, \text{Copies_Sold})}(\text{BookID}, \text{AuthorName} \bowtie \text{SUM Quantity}(\text{ORDERS}))$

$\rho_{\text{MOST_COPIES}(\text{AuthorName}, \text{Copies_Sold})}(\text{AuthorName} \bowtie \text{MAX Copies_Sold}(\text{AGGREGATE_DETAILS}))$

$\pi_{\text{AuthorName}, \text{Copies_Sold}}((\sigma_{\text{Copies_Sold} = \text{MOST_COPIES.Copies_Sold}}(\text{AGGREGATE_DETAILS}))$

SQL Query:

```
SELECT Books_Author.AuthorName, SUM(Quantity) as Copies_Sold
FROM Books_Author NATURAL JOIN Shopping_Cart
GROUP BY BookID, AuthorName
ORDER BY Copies_Sold DESC;
```

15. Find the most profitable author in the database for the bookstore. Return author name and total profit produced by the author

Relational Algebra:

ORDERS $\leftarrow$ BOOKS_AUTHOR * SHOPPING_CART

$\rho_{\text{AGGREGATE_DETAILS}}(\text{BookID}, \text{AuthorName}, \text{Profit_Generated}) (\text{BookID}, \text{AuthorName} \bowtie \text{SUM Cost (ORDERS)})$
 $\rho_{\text{MOST_PROFITABLE}}(\text{Profit_Generated}) (\bowtie \text{MAX Profit_Generated (AGGREGATE_DETAILS)})$
 $\pi_{\text{AuthorName}, \text{Profit_Generated}} (\sigma \text{Profit_Generated} = \text{MOST_PROFITABLE.Profit_Generated (AGGREGATE_DETAILS)})$

SQL Query:

```
SELECT Books_Author.AuthorName, SUM(Cost) as Profit_Generated
FROM Books_Author NATURAL JOIN Shopping_Cart
GROUP BY BookID, AuthorName
ORDER BY Profit_Generated DESC;
```

16. Provide the list of customers who purchased anything written by the most profitable author in the database

Relational Algebra:

$\text{ORDERS} \leftarrow \text{BOOKS_AUTHOR} * \text{SHOPPING_CART}$
 $\text{CUSTOMER_DETAILS} \leftarrow \text{ORDERS} \bowtie_{\text{ORDERS.CustomerID} = \text{CUSTOMER.CustomerID}} (\text{CUSTOMER})$
 $\text{AUTHOR_DETAILS} \leftarrow \text{CUSTOMER_DETAILS} \bowtie_{\text{CUSTOMER_DETAILS.BookID} = \text{Books_Author.BookID}} (\text{BOOKS_AUTHOR})$
 $\rho_{\text{AGGREGATE_DETAILS}}(\text{BookID}, \text{AuthorName}, \text{Profit_Generated}) (\text{BookID}, \text{AuthorName} \bowtie \text{SUM Cost (ORDERS)})$
 $\rho_{\text{MOST_PROFITABLE}}(\text{Profit_Generated}) (\bowtie \text{MAX Profit_Generated (AGGREGATE_DETAILS)})$
 $\text{AUTHOR} \leftarrow \sigma_{\text{Profit_Generated} = \text{MOST_PROFITABLE.Profit_Generated}} (\text{AGGREGATE_DETAILS})$
 $\text{RESULTS} \leftarrow \pi_{\text{FirstName}, \text{LastName}, \text{Email}} (\sigma \text{BookID} = \text{AUTHOR.BookID (AUTHOR_DETAILS)})$

SQL Query:

```
SELECT Customer.FirstName, Customer.LastName, Customer.Email
FROM Shopping_Cart NATURAL JOIN Customer_Orders
JOIN Customer ON Customer.CustomerID = Customer_Orders.CustomerID
JOIN Books_Author ON Shopping_Cart.BookID = Books_Author.BookID
WHERE Books_Author.AuthorName IN
    (SELECT Books_Author.AuthorName
     FROM Books_Author NATURAL JOIN Shopping_Cart
     GROUP BY BookID, AuthorName
     ORDER BY SUM(Cost) DESC
     LIMIT 1);
```

17. Provide the list of authors who wrote the books purchased by the customers who have spent more than the average customer

Relational Algebra:

$\text{ORDERS} \leftarrow \text{SHOPPING_CART} * \text{CUSTOMER_ORDERS}$
 $\rho_{\text{AMOUNT_SPENT}}(\text{CustomerID}, \text{Total_Expenditure}) (\text{CustomerID} \bowtie \text{SUM Cost (ORDERS)})$
 $\rho_{\text{AVERAGE_EXPENDITURE}}(\text{Average}) (\bowtie \text{AVG Total_Expenditure (AMOUNT_SPENT)})$
 $\text{CUSTOMERS} \leftarrow \pi_{\text{CustomerID}, \text{FirstName}, \text{LastName}, \text{Email}, \text{Total_Expenditure}} (\sigma \text{Total_Expenditure} > \text{AVERAGE_EXPENDITURE.Average (AMOUNT_SPENT * CUSTOMER)})$
 $\text{AUTHORS} \leftarrow \text{BOOKS_AUTHOR} \bowtie_{\text{BOOKS_AUTHOR.BookID} = \text{ORDERS.BookID}} (\text{ORDERS})$
 $\text{RESULTS} \leftarrow \pi_{\text{AuthorName}} (\text{AUTHORS} \bowtie_{\text{AUTHORS.CustomerID} = \text{CUSTOMERS.CustomerID}} (\text{CUSTOMERS}))$

SQL Query:

```

SELECT Books_Author.AuthorName
FROM Shopping_Cart NATURAL JOIN Customer_Orders
JOIN Books_Author ON Books_Author.BookID = Shopping_Cart.BookID
WHERE CustomerID IN
    (SELECT Customer_Orders.CustomerID
     FROM Shopping_Cart NATURAL JOIN Customer_Orders
     JOIN Customer ON Customer_Orders.CustomerID = Customer.CustomerID
     GROUP BY Customer_Orders.CustomerID
     HAVING SUM(Cost) >
        (SELECT AVG(TotalSpent)
         FROM (SELECT SUM(Cost) AS TotalSpent
              FROM Shopping_Cart NATURAL JOIN Customer_Orders
              GROUP BY Customer_Orders.CustomerID)))
);

```

Insert and Delete Syntax

1. Adding/Deleting new Books:

Sample Insert Statement:

```

INSERT INTO Book (BookID, PublisherID, InventoryID, Title, Year, Genre, ISBN, Price)
VALUES (501, 23, 110, 'Sample Book' , 2002, 'Computer', '999999999', 100);

```

Sample Delete Statement:

```

-- Step 1: Update Books_Author
DELETE FROM Books_Author
WHERE BookID = 501;

```

```

-- Step 2: Update Shopping_Cart
DELETE FROM Shopping_Cart
WHERE BookID = 501;

```

```

-- Step 3: Update Inventory_Order
DELETE FROM Inventory_Order
WHERE BookID = 501;

```

```

--Step 4: Update Book
DELETE FROM Book
WHERE BookID = 501;

```

2. Adding/Deleting new Publishers

Adding publishers to the database does not have any dependencies. A new publisher can be added to the database regardless of whether they have published a book.

Sample Insert Statement:

```

INSERT INTO Publishers (PublisherID, PName)
VALUES (101, 'HarperCollins');

```

Sample Delete Statement:

-- Step 1: Update Book relation (Assume new Publisher is 23)
UPDATE Book SET PublisherID = 23 WHERE PublisherID = 101

-- Step 2: Update Supplies relation
DELETE FROM Supplies
WHERE PublisherID = 101

-- Step 3: Update Publishers relation
DELETE FROM Publishers
WHERE PublisherID = 101;

3. Adding/Deleting new Authors

To add a author to the database, it needs to be made sure that they are referencing a book that is present in the BOOK relation. Therefore, if we are adding a author for a book with BookID = 501

INSERT INTO Book (BookID, PublisherID, InventoryID, Title, Year, Genre, ISBN, Price)
VALUES (501, 23, 110, 'Sample Book' , 2002, 'Computer', '999999999', 100);

Sample Insert Statement for an Author:

INSERT INTO Books_Author (BookID, AuthorName)
VALUES (501,'Sample Author');

Sample Delete Statement for an Author:

DELETE FROM Books_Author
WHERE BookID = 501 AND AuthorName = 'Sample Author';

4. Adding/Deleting new Users

Sample Insert Statement for a Customer:

INSERT INTO Customer (CustomerID, FirstName, LastName, Email, Phone_Number)
VALUES (1001, 'Alice', 'Smith', 'alice@sample.com', '555-234-1234');

Sample Delete Statement for a Customer:

-- Step 1: Update Customer_Orders relation
DELETE FROM Customer_Orders
WHERE CustomerID = 1001;

-- Step 2: Update Supplies relation
DELETE FROM Reviews
WHERE CustomerID = 1001

-- Step 3: Update Customer relation
DELETE FROM Customer
WHERE CustomerID = 1001;

Sample Insert Statement for a Admin:

**INSERT INTO Admin (AdminID, FirstName, LastName, Email)
VALUES (1001, 'Alice', 'Smith', 'alice@sample.com')**

Sample Delete Statement for a Admin:

-- Step 1: Update Admin_Orders relation

**DELETE FROM Admin_Orders
WHERE AdminID = 1001;**

-- Step 3: Update Admin relation

**DELETE FROM Admin
WHERE AdminID = 1001;**

5. Adding/Deleting Reviews

Sample Insert Statement for a Review:

**INSERT INTO Reviews (Review_No, CustomerID, Rating, Feedback, Date)
VALUES (1, 1001, 5, 'Excellent read!', '2025-04-19');**

Sample Delete Statement for a Review:

**DELETE FROM Reviews
WHERE Review_No = 1 AND CustomerID = 1001;**

6. Adding/Deleting Orders

For all insert and delete statement it needs to be made sure that appropriate customer and admin ids exist in the CUSTOMER and ADMIN relation

Sample Insert Statement for a Customer Order:

-- Step 1: Insert into Shopping_Cart

**INSERT INTO Shopping_Cart (Order_No, BookID, Order_Date, Quantity, Cost)
VALUES (2001, 501, '2025-04-19', 2, 29.98);**

-- Step 2: Insert into Customer_Orders

**INSERT INTO Customer_Orders (Order_No, CustomerID)
VALUES (2001, 1001);**

Sample Delete Statement for a Customer Order:

-- Step 1: Delete from Shopping_Cart

**DELETE FROM Shopping_Cart
WHERE Order_No = 2001;**

Sample Insert Statement for an Inventory Order

-- Step 1: Insert into Inventory_Order

**INSERT INTO Inventory_Order (Order_No, BookID, Order_Date, Quantity, Cost)
VALUES (2001, 501, '2025-04-19', 2, 29.98);**

-- Step 2: Insert into Admin_Orders

```
INSERT INTO Admin_Orders (Order_No, AdminID)  
VALUES (2001, 1001);
```

-- Step 3: Insert into Supplies

```
INSERT INTO Supplies (Order_No, PublisherID)  
VALUES (2001, 23);
```

Sample Delete Statement for an Inventory Order:

-- Step 1: Delete from Supplies

```
DELETE FROM Supplies  
WHERE Order_No = 2001;
```

-- Step 3: Delete from Shopping_Cart

```
DELETE FROM Shopping_Cart  
WHERE Order_No = 2001;
```

SECTION III: Checkpoint Documents

Appendix A: Project Checkpoint 1

In a **NEATLY TYPED** document, provide the following:

1. Based on the requirements given in the project overview, list the entities to be modeled in this database. For each entity, provide a list of associated attributes.

- **ADMIN(AdminID, FName, LName, Email)**
- **BOOK(BookID, ISBN, Title, Author(s), Publisher, Year, Price, Category)**
- **INVENTORY(BookID, Quantity)**
- **CUSTOMER(CustomerID, EmailID, Name, Phone_Number, PurchasedBooks)**

2. Based on the requirements given in the project overview, what are the various relationships between entities? (For example, "CUSTOMER entities purchase BOOK entities").

- **CUSTOMER entities purchase BOOK entities**
- **ADMIN entity stocks INVENTORY entities**
- **INVENTORY entities contain BOOK entities**

3. Propose at least two additional entities that it would be useful for this database to model beyond the scope of the project requirements. Provide a list of possible attributes for the additional entities and possible relationships they may have with each other and the rest of the entities in the database. Give a brief, one sentence rationale for why adding these entities would be interesting/useful to the stakeholders for this database project.

- **REVIEWS(ReviewNo, Rating, Feedback, Date)**

Rationale: This entity helps stakeholders keep track of feedback from customers about the database which is important for making improvements in the database.

Relations: CUSTOMER entities Write REVIEWS entities.

- **SHOPPING_CART(OrderNo, BookID, Quantity, Cost, Purchase_Date)**

Rationale: Customers can order multiple books and save details for their order to change at a later time.

Relations: CUSTOMER entities create SHOPPING CART entities.

- **PUBLISHERS(PublisherID, Pname, BooksPublished)**

Rationale: Publishers should be able to add new books to the bookstore.

Relations: PUBLISHER entities publish BOOK entities

- **INVENTORY_ORDERS(Order No., Order Date, AdminID, BookID, Quantity, Cost)**

Rationale: Administrators of the bookstore keeping track of the quantities of books should be able to create orders to restock inventory. These orders are fulfilled by the publishers of books

Relations:

1. ADMIN entities create INVENTORY_ORDER entities
2. PUBLISHER entities supply books for INVENTORY_ORDER entities

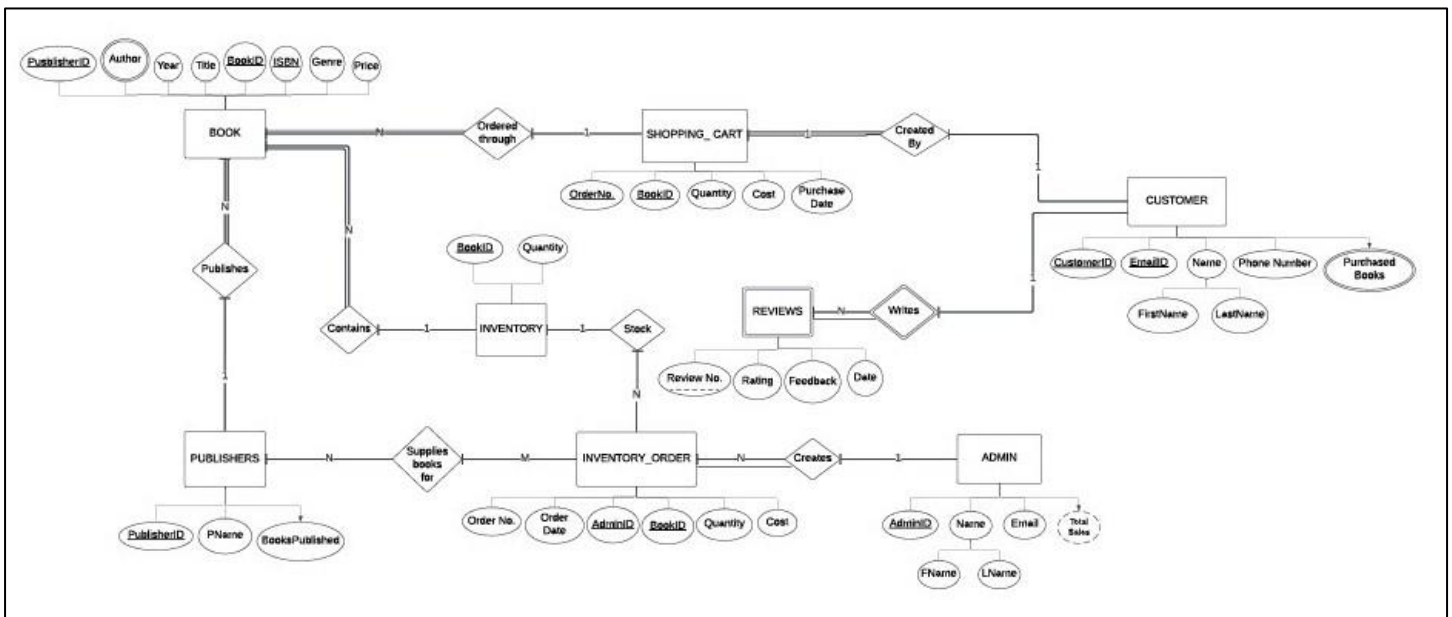
4. Give at least four examples of some informal queries/reports that it might be useful for this database might be used to generate. Include one example for each of the additional entities you proposed in question 3 above.
- **Insert new <Publisher> into database**
 - **Insert new <Book> into database**
 - **Get quantity of a particular <Book> in the <Inventory>**
 - **Check the number of books under each <Genre>**
 - **Check all <purchased books> of <CustomerID>**
 - **Find all books published by a <Publisher>**
 - **Add new <Book> to a <Shopping Cart>**
 - **Add new <Review> for the bookstore**
 - **Insert new <Customer> as a member of the bookstore**
 - **Get order details like Books ordered and their quantities for stocking order using a <Inventory_Order> entity**
 - **Calculate total sales using the <Cost> attribute of a <Shopping_Cart> entity**
5. Suppose we want to add a new publisher to the database. How would we do that given the entities and relationships you've outlined above? Given your above description, is it possible to add a new publisher to your database without knowing the title of any books they have published? If not, revise your model to allow for publishers to be added as separate entities.

The current database model contains a PUBLISHER entity. Since, it has been added as a separate entity, it is possible to enter a new publisher into the database without knowing the books that they published.

6. Determine at least three other informal update operations and describe what entities would need to have attributes altered and how they would need to be changed given your above descriptions. Include one example for each of the additional entities you proposed in question 3 above.
- **Update quantity of a certain book in their shopping cart. The Quantity attribute in the SHOPPING_CART will need to be increased or decreased**
 - **Update quantity of a particular book in the inventory. The Quantity attribute in the INVENTORY entity will need to be increased or decreased.**
 - **Update the order details for an inventory stock order: change books ordered, quantity of books ordered, and the date on which they are ordered. Attributes affected include BookID, Quantity, and Order Date of the INVENTORY_ORDER entities. BookIDs may be removed, and quantities of books can be increased or decreased.**
 - **Update quantity of books published by a publisher. The BooksPublished attribute in the PUBLISHER entity will need to be increased.**
 - **Change rating for a customer review. The Rating attribute in the REVIEW entity will need to be increased or decreased.**
 - **Update customer review. The Feedback attribute in the REVIEW entity will need to be updated.**

- Update Customer emailID or phone number. The EmailID/PhoneNumber of the CUSTOMER entity will need to be changed.
- Update Administrator email or name. The EmailID/Name attribute of the ADMIN entity will need to be changed.
- Update the price of a book for offers or discounts. The Price attribute of the BOOK entity will need to be changed

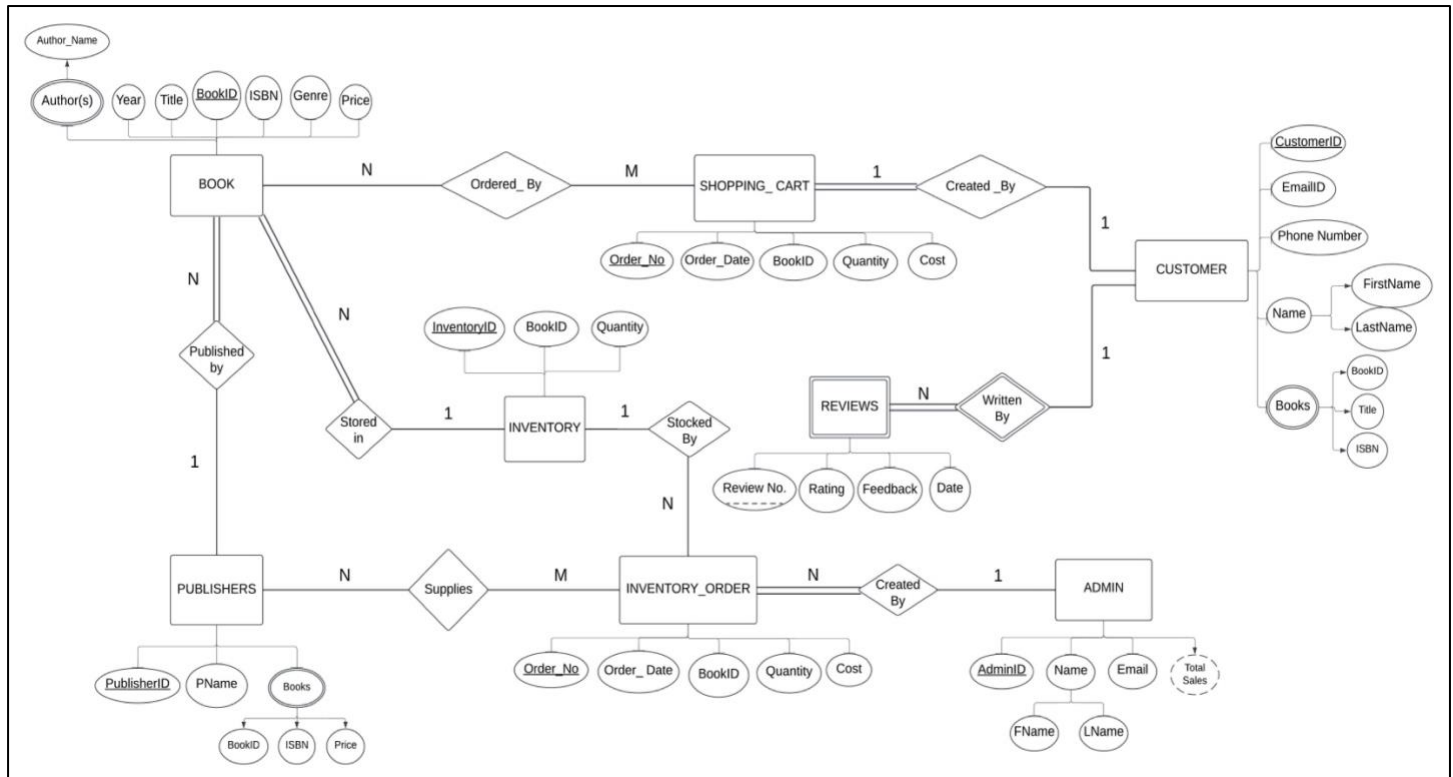
7. Provide an ER diagram for your database. Make sure you include all of the entities and relationships you determined in the questions above **INCLUDING the entities for question 3 above**, and remember that **EVERY** entity in your model needs to connect to another entity in the model via some kind of relationship.



Appendix B: Project Checkpoint 2

In a **NEATLY TYPED** document, provide the following:

1. Provide a current version of your ER Model as per Project Checkpoint 01. If you were instructed to change the model for Project Checkpoint 01, make sure you use the revised version of your ER Model.



2. Map your ER model to a relational schema. Indicate all primary and foreign keys.

ADMIN(AdminID#, FirstName, LastName, Email)

INVENTORY_ORDER(Order_No#, Order_Date, InventoryID^(FK), AdminID^(FK), BookID^(FK), Quantity, Cost)

BOOK(BookID#, PublisherID^(FK), InventoryID^(FK), Title, Year, Genre, ISBN, Price)

ORDERED_BY(Order_No#, BookID^(FK))

PUBLISHERS(PublisherID#, PName)

SUPPLIES(Order_No#, PublisherID^(FK), Rating, Feedback, Date)

AUTHOR(BookID#^(FK), Author_Name)

INVENTORY(InventoryID#, BookID^(FK), Quantity)

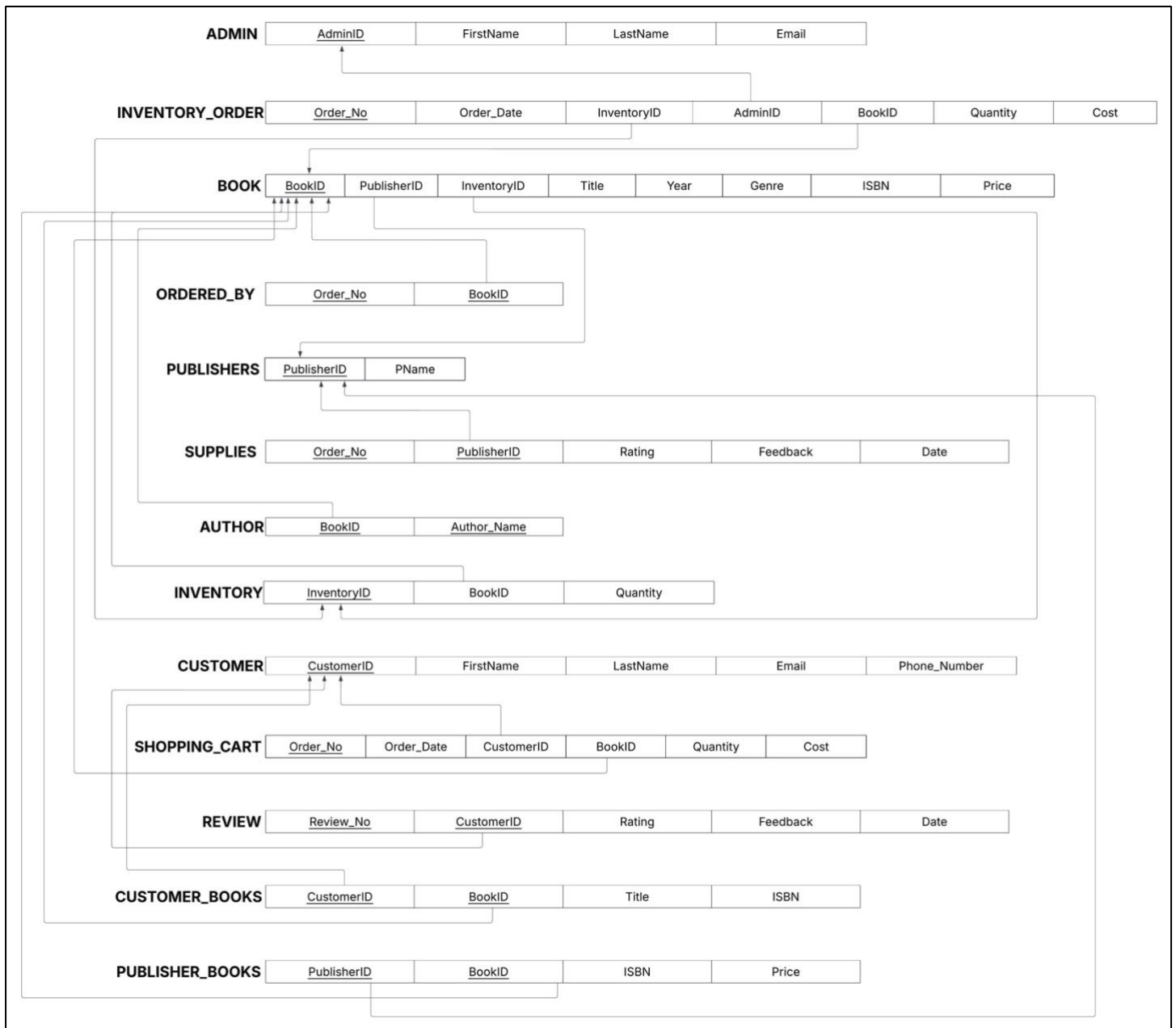
CUSTOMER(CustomerID#, FirstName, LastName, Email, Phone_Number)

SHOPPING_CART(Order_No#, Order_Date, CustomerID^(FK), BookID^(FK), Quantity, Cost)

REVIEW(Review_No#, CustomerID^(FK), Rating, Feedback, Date)

CUSTOMER_BOOKS(CustomerID#^(FK), BookID#^(FK), Title, ISBN)

PUBLISHER_BOOKS(PublisherID#^(FK), BookID#^(FK), ISBN, Price)



3. Given your relational schema, provide the relational algebra to perform the following queries. If your schema cannot provide answers to these queries, revise your ER Model and your relational schema to contain the appropriate information for these queries:

a. Find the titles of all books by Pratchett that cost less than \$10

PRATCHETT_ID $\leftarrow \pi_{\text{BookID}} (\sigma_{\text{Author_Name} = \text{'Pratchett'}} (\text{AUTHOR}))$
BOOKS $\leftarrow \text{PRATCHETT_ID} \bowtie_{\text{PRATCHETT_ID.BookID} = \text{BOOK.BookID}} (\text{BOOK})$
RESULT $\leftarrow \pi_{\text{Title}} (\sigma_{\text{Price} < 10} (\text{BOOKS}))$

b. Give all the titles and their dates of purchase made by a single customer (Assumption CustomerID = 100)

BOOK_TITLES $\leftarrow \pi_{\text{BookID}, \text{Title}} (\sigma_{\text{CustomerID} = 100} (\text{CUSTOMER_BOOKS}))$
CUSTOMER_ORDERS $\leftarrow \sigma_{\text{CustomerID} = 100} (\text{SHOPPING_CART})$

PURCHASE_DATES $\leftarrow$ **BOOK_TITLES** $\bowtie$ **BOOK_TITLES.BookID = SHOPPING_CART.BookID**
(CUSTOMER_ORDERS)
RESULT $\leftarrow \pi_{\text{Title, Order_Date}}$ (**PURCHASE_DATES**)

- c. Find the titles and ISBNs for all books with less than 5 copies in stock

LOW_BOOK_IDS $\leftarrow \pi_{\text{BookID}} (\sigma_{\text{Quantity} < 5} (\text{INVENTORY}))$
BOOKS $\leftarrow$ **LOW_BOOK_IDS** $\bowtie$ **LOW_BOOK_IDS.BookID = BOOK.BookID** (**BOOK**)
RESULT $\leftarrow \pi_{\text{Title, ISBN}}$ (**BOOKS**)

- d. Give all the customers who purchased a book by Pratchett and the titles of Pratchett books they purchased

PRATCHETT_BOOK_IDS $\leftarrow \pi_{\text{BookID}} (\sigma_{\text{Author_Name} = \text{'Pratchett'}} (\text{AUTHOR}))$
PRATCHETT_CUST $\leftarrow$ **PRATCHETT_BOOK_IDS** $\bowtie$ **PRATCHETT_BOOK_IDS.BookID =**
CUSTOMER_BOOKS.BookID (**CUSTOMER_BOOKS**)
CUSTOMERS_ENHANCED $\leftarrow$ **PRATCHETT_CUST** $\bowtie$ **PRATCHETT_CUST.CustomerID =**
CUSTOMER.CustomerID (**CUSTOMER**)
RESULT $\leftarrow \pi_{\text{CustomerID, FirstName, LastName, Title}}$ (**CUSTOMERS_ENHANCED**)

- e. Find the total number of books purchased by a single customer (Assumption CustomerID = 100)

CUSTOMER_BOOK_COUNT $\leftarrow$ **CustomerID** $\bowtie$ **COUNT BookID** (**CUSTOMER_BOOKS**)
RESULT $\leftarrow \sigma_{\text{CustomerID} = 100}$ (**CUSTOMERS_BOOK_COUNT**)

- f. Find the customer who has purchased the most books and the total number of books they have purchased

$\rho_{\text{BOOK_COUNT}(\text{CustomerID, BookCount})}$ (**CustomerID** $\bowtie$ **COUNT BookID** (**CUSTOMER_BOOKS**))
 $\rho_{\text{MAX_COUNT}(\text{CustomerID, NumBooks})}$ (**CustomerID** $\bowtie$ **MAX BookCount** (**BOOK_COUNT**))
RESULT $\leftarrow \pi_{\text{CustomerID, FirstName, LastName, NumBooks}}$ (**MAX_COUNT** $\bowtie$ **MAX_COUNT.CustomerID =**
CUSTOMER.CustomerID (**CUSTOMER**))

4. Come up with three additional interesting queries that your database can provide. Give what the queries are supposed to retrieve in plain English and then as relational algebra. Your queries should include joins and at least one should include an aggregate function. At least one of your queries should use “extra” entities you added to your model in Checkpoint 01.

- a. Find count of ratings per Customer and average rating per Customer along with Customer name and email

$\rho_{\text{REVIEW_PER_CUST}(\text{CustomerID, NumReview, AvgRating})}$ (**CustomerID** $\bowtie$ **COUNT ReviewNo, AVERAGE Rating** (**REVIEWS**))
CUSTOMER_DETAILS $\leftarrow$ **REVIEW_PER_CUST** $\bowtie$ **REVIEW_PER_CUST.CustomerID = CUSTOMER.CustomerID**
(CUSTOMER)
RESULT $\leftarrow \pi_{\text{CustomerID, FirstName, LastName, NumReview, AvgRating}}$ (**CUSTOMER_DETAILS**)

- b. Find all stock orders made for books by ‘Pratchett’ and retrieve admin details, date of order, quantity, and cost

PRATCHETT_BOOK_IDS $\leftarrow \pi_{\text{BookID}} (\sigma_{\text{Author_Name} = \text{'Pratchett'}} (\text{AUTHOR}))$

PRATCHETT_ORDERS $\leftarrow$ **PRATCHETT_BOOK_IDS** $\bowtie$ **PRATCHETT_BOOK_IDS**.BookID =
INVENTORY_ORDER.BookID (**INVENTORY_ORDER**)
ADMIN_DETAILS $\leftarrow$ **PRATCHETT_ORDERS** $\bowtie$ **PRATCHETT_ORDERS**.AdminID = **ADMIN**.AdminID (**ADMIN**)
RESULT $\leftarrow$ π AdminID, FName, LName, Order_Date, Quantity, Cost (**ADMIN_DETAILS**)

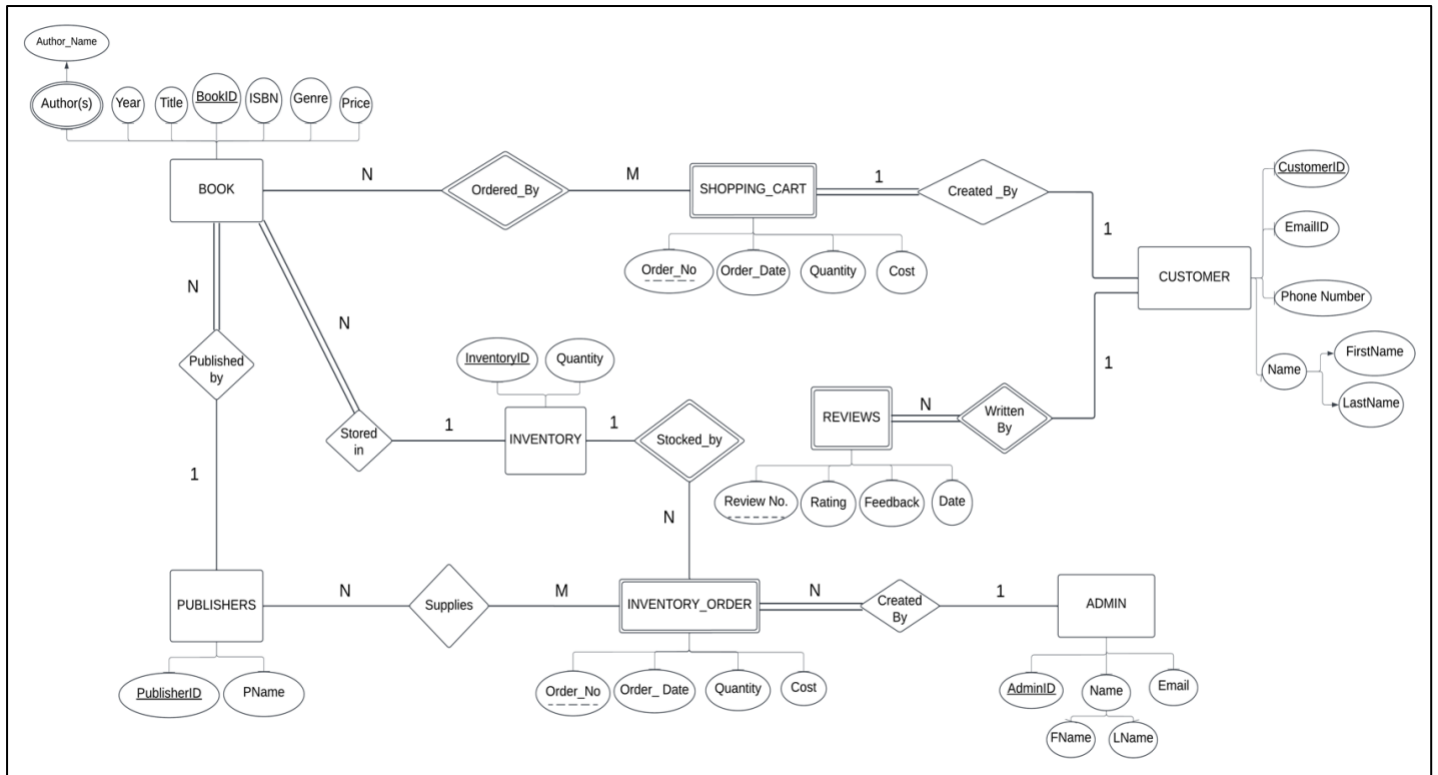
- c. Find all publishers for books authored by 'Pratchett' along with book titles and price

PRATCHETT_BOOK_IDS $\leftarrow$ π BookID (σ Author_Name = 'Pratchett' (**AUTHOR**))
PRATCHETT_BOOKS $\leftarrow$ π BookID, PublisherID, Title, Price (**PRATCHETT_BOOKS_IDS** $\bowtie$
PRATCHETT_BOOK_IDS.BookID = **BOOK**.BookID (**BOOK**))
PRATCHETT_PUB $\leftarrow$ **PRATCHETT_BOOKS** $\bowtie$ **PRATCHETT_BOOKS**.PublisherID =
PUBLISHER_BOOKS.PublisherID (**PUBLISHERS**)
RESULT $\leftarrow$ π PublisherID, PName, Title, Price (**PRATCHETT_PUB**)

Appendix C: Project Checkpoint 3:

1. Provide a current version of your ER Diagram and Relational Model as per Project Checkpoint 02.
02. If you were instructed to change the model for Project Checkpoint 02, make sure you use the revised versions of your models

ER Diagram:



Relational Model:

ADMIN(AdminID#, FirstName, LastName, Email)

INVENTORY_ORDER(Order_No#, Order_Date, InventoryID^(FK), AdminID^(FK), BookID^(FK), Quantity, Cost)

BOOK(BookID#, PublisherID^(FK), InventoryID^(FK), Title, Year, Genre, ISBN, Price)

PUBLISHERS(PublisherID#, PName)

SUPPLIES(Order_No^(FK), PublisherID^(FK))

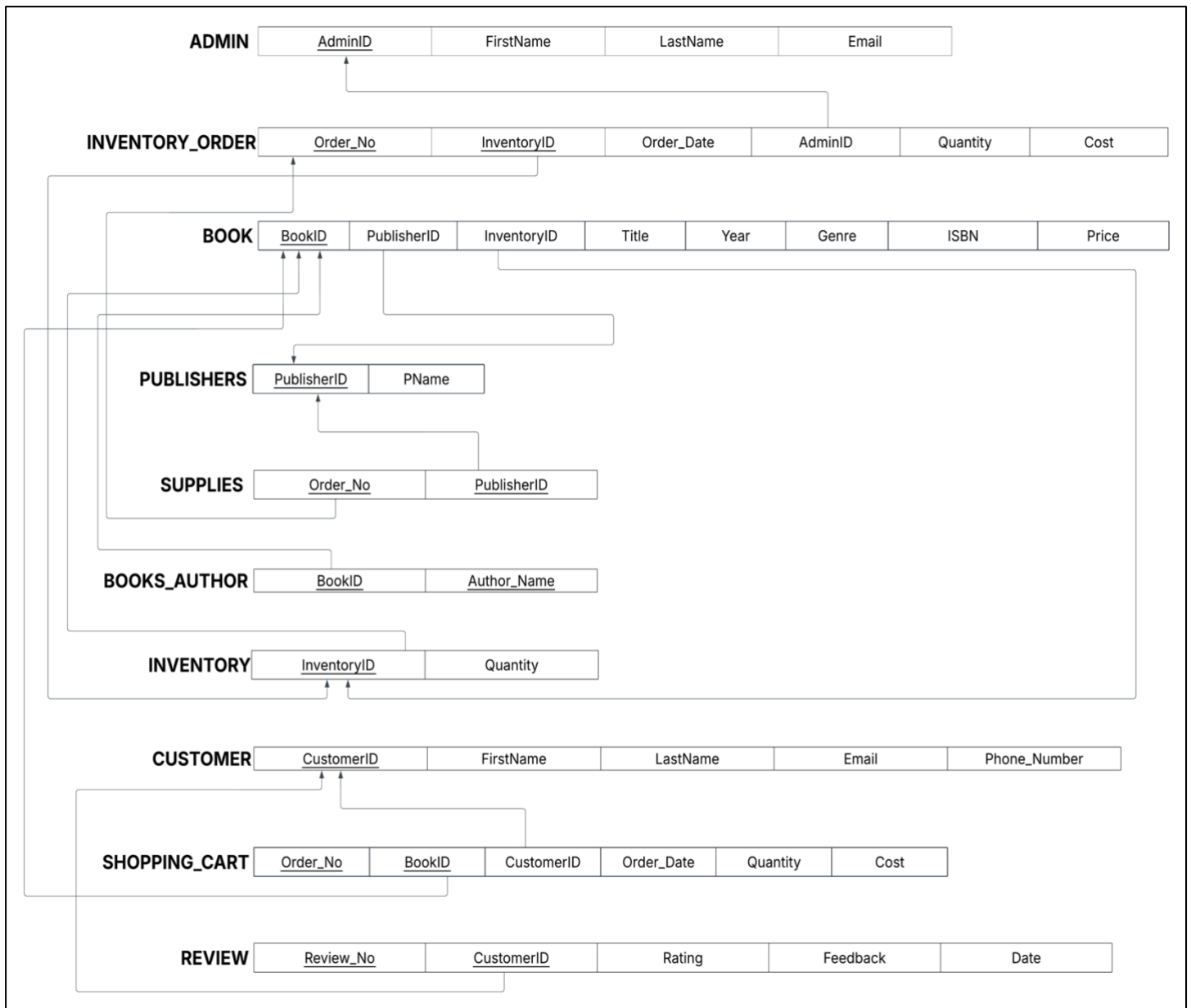
BOOKS_AUTHOR(BookID#^(FK), Author_Name)

INVENTORY(InventoryID#, Quantity)

CUSTOMER(CustomerID#, FirstName, LastName, Email, Phone_Number)

SHOPPING_CART(Order_No#, Order_Date, CustomerID^(FK), BookID^(FK), Quantity, Cost)

REVIEW(Review_No#, CustomerID^(FK), Rating, Feedback, Date)



- Given your relational schema, create a text file containing the SQL code to create your database schema. Use this SQL to create a database in SQLite. Populate this database with the data provided for the project as well as 20 sample records for each table that does not contain data provided in the original project documents.

Bookstore.txt contains the create table statements

- Given your relational schema, provide the SQL to perform the following queries. If your schema cannot provide answers to these queries, revise your ER Model and your relational schema to contain the appropriate information for these queries. These queries should be provided in a plain text file named "WorksheetTwoSimpleQueries.txt":

- a. Find the titles of all books by Pratchett that cost less than \$10

```
SELECT BookID, Book.Title
FROM Book NATURAL JOIN Books_Author
WHERE Books_Author.AuthorName LIKE '%Pratchett%' AND Book.Price < 10;
```

- b. Give all the titles and their dates of purchase made by a single customer (you choose how to designate the customer)

```
SELECT Book.Title, Shopping_Cart.Order_Date
FROM Book NATURAL JOIN Shopping_Cart
WHERE Shopping_Cart.CustomerID = 5;
```

- c. Find the titles and ISBNs for all books with less than 5 copies in stock

```
SELECT Book.Title, Book.ISBN
FROM Book JOIN Inventory ON Book.InventoryID = Inventory.InventoryID
WHERE Inventory.Quantity < 5;
```

- d. Give all the customers who purchased a book by Pratchett and the titles of Pratchett books they purchased

```
SELECT Customer.CustomerID, Customer.FirstName, Customer.LastName, Book.Title
FROM Books_Author NATURAL JOIN Book
JOIN Shopping_Cart ON Book.BookID = Shopping_Cart.BookID
JOIN Customer ON Shopping_Cart.CustomerID = Customer.CustomerID
WHERE Books_Author.AuthorName LIKE '%Pratchett%';
```

- e. Find the total number of books purchased by a single customer (you choose how to designate the customer)

```
SELECT CustomerID, Book_Count
FROM (SELECT CustomerID, COUNT(*) AS Book_Count
      FROM Shopping_Cart
      GROUP BY CustomerID)
WHERE CustomerID = 1;
```

- f. Find the customer who has purchased the most books and the total number of books they have purchased

```
SELECT CustomerID, MAX(Book_Count) AS Max_Count
FROM (SELECT CustomerID, COUNT(*) AS Book_Count
      FROM Shopping_Cart
      GROUP BY CustomerID);
```

4. For Project Checkpoint 02, you were asked to come up with three additional interesting queries that your database can provide. Give what those queries are supposed to retrieve in plain English, as relational algebra and then as SQL. Your queries should include joins and at least one should include an aggregate function, and they should be the same as the queries you outlined for Worksheet 02. If you were instructed to fix the queries in Checkpoint 02, make sure

you use the fixed queries here. These queries should be provided in a plain text file named “WorksheetTwoExtraQueries.txt”.

- a. Find count of ratings per Customer and average rating per Customer along with Customer name and email

```
SELECT Customer.FirstName, Customer.LastName, COUNT(Rating) AS Rating_Count,
AVG(Rating) as Avg_Rating, Customer.Email
FROM Reviews JOIN Customer ON Reviews.CustomerID = Customer.CustomerID
GROUP BY Reviews.CustomerID;
```

- b. Find all stock orders made for books by ‘Pratchett’ and retrieve admin details, date of order, quantity, and cost

```
SELECT Admin.FirstName, Admin.LastName, Book.Title, Inventory_Order.InventoryID,
Inventory_Order.Order_Date, Inventory_Order.Quantity, Inventory_Order.Cost
FROM Inventory_Order JOIN Book ON Inventory_Order.InventoryID = Book.InventoryID
JOIN ADMIN ON Inventory_Order.AdminID = ADMIN.AdminID
JOIN Books_Author ON Book.BookID = Books_Author.BookID
WHERE Books_Author.AuthorName LIKE '%Pratchett%';
```

- c. Find all publishers for books authored by ‘Pratchett’ along with book titles and price

```
SELECT Publishers.PublisherID, Publishers.Pname, Book.Title, Book.Price
FROM Publishers NATURAL JOIN Book
JOIN Books_Author ON Book.BookID = Books_Author.BookID
WHERE Books_Author.AuthorName LIKE '%Pratchett%';
```

5. Given your relational schema, provide the SQL for the following more advanced queries. These queries may require you to use techniques such as nesting, aggregation using having clauses, and other techniques. If your database schema does not contain the information to answer to these queries, revise your ER Model and your relational schema to contain the appropriate information for these queries. **Note that if your database does contain the information but in non-aggregated form, you should NOT revise your model but instead figure out how to aggregate it for the query!** These queries should be provided in a plain text file named “WorksheetTwoAdvancedQueries.txt”.

- a. Provide a list of customer names, along with the total dollar amount each customer has spent.

```
SELECT Customer.FirstName, Customer.LastName, Sum(Cost) as Amount_Spent
FROM Customer JOIN Shopping_Cart ON Customer.CustomerID = Shopping_Cart.CustomerID
GROUP BY Customer.CustomerID;
```

- b. Provide a list of customer names and e-mail addresses for customers who have spent more than the average customer.

```
SELECT Shopping_Cart.CustomerID, Customer.FirstName, Customer.LastName,
Customer.Email, SUM(Cost) as Total_Expenditure
FROM Shopping_Cart
JOIN Customer ON Shopping_Cart.CustomerID = Customer.CustomerID
```

```
GROUP BY Shopping_Cart.CustomerID
HAVING Total_Expenditure >
    (SELECT AVG(TotalSpent) FROM
    (SELECT SUM(Cost) AS TotalSpent
    FROM Shopping_Cart
    GROUP BY CustomerID));
```

- c. Provide a list of the titles in the database and associated total copies sold to customers, sorted from the title that has sold the most individual copies to the title that has sold the least.

```
SELECT BookID, Book.Title, SUM(Quantity) as Copies_Sold
FROM Book NATURAL JOIN Shopping_Cart
GROUP BY BookID
ORDER BY Copies_Sold DESC;
```

- d. Provide a list of the titles in the database and associated dollar totals for copies sold to customers, sorted from the title that has sold the highest dollar amount to the title that has sold the smallest.

```
SELECT BookID, Book.Title, SUM(Cost) as Dollar_Total
FROM Book NATURAL JOIN Shopping_Cart
GROUP BY BookID
ORDER BY Dollar_Total DESC;
```

- e. Find the most popular author in the database (i.e. the one who has sold the most books)

```
SELECT Books_Author.AuthorName, SUM(Quantity) as Copies_Sold
FROM Books_Author NATURAL JOIN Shopping_Cart
GROUP BY BookID, AuthorName
ORDER BY Copies_Sold DESC;
```

- f. Find the most profitable author in the database for this store (i.e. the one who has brought in the most money)

```
SELECT Books_Author.AuthorName, SUM(Cost) as Profit_Generated
FROM Books_Author NATURAL JOIN Shopping_Cart
GROUP BY BookID, AuthorName
ORDER BY Profit_Generated DESC;
```

- g. Provide a list of customer information for customers who purchased anything written by the most profitable author in the database.

```
SELECT Customer.FirstName, Customer.LastName, Customer.Email
FROM Customer JOIN Shopping_Cart ON Customer.CustomerID = Shopping_Cart.CustomerID
JOIN Books_Author ON Shopping_Cart.BookID = Books_Author.BookID
WHERE Books_Author.AuthorName IN
    (SELECT Books_Author.AuthorName
    FROM Books_Author NATURAL JOIN Shopping_Cart
    GROUP BY BookID, AuthorName
    ORDER BY SUM(Cost) DESC
    LIMIT 1);
```

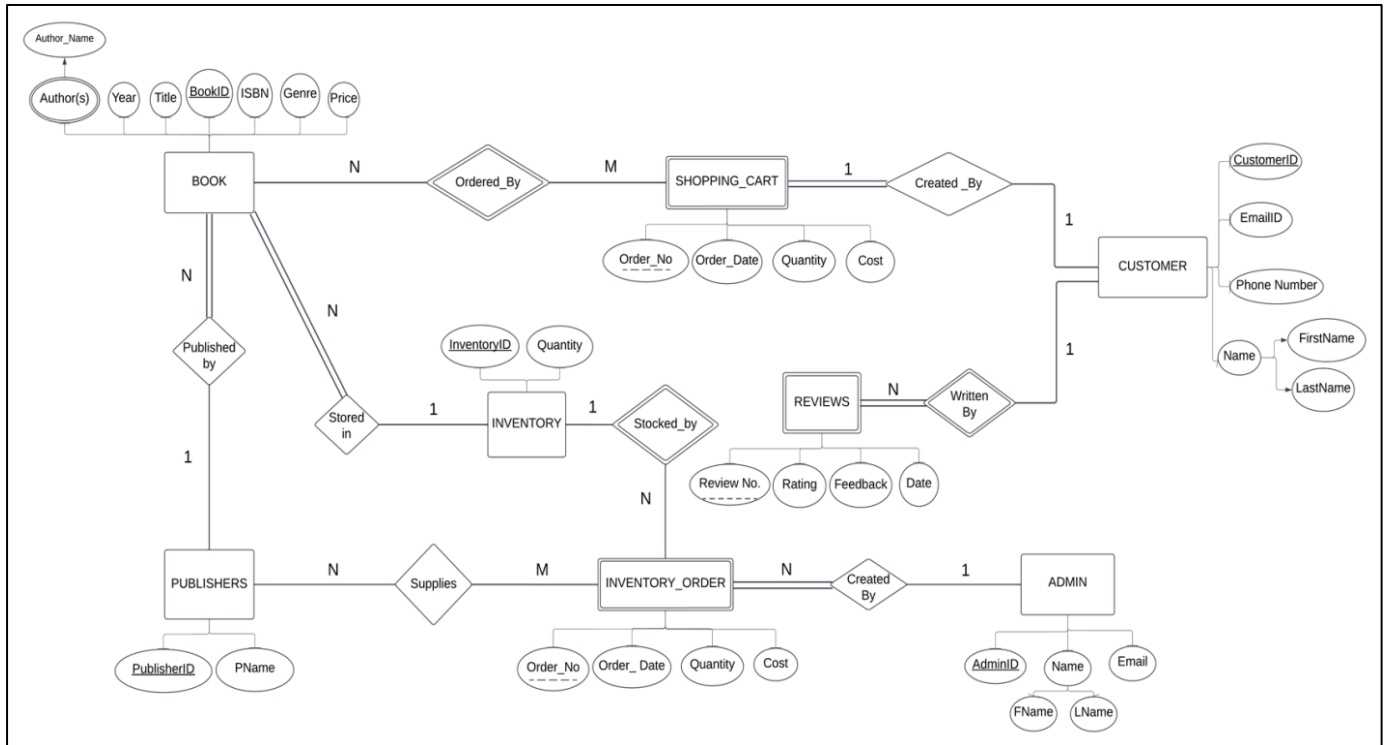
- h. Provide the list of authors who wrote the books purchased by the customers who have spent more than the average customer.

```
SELECT Books_Author.AuthorName
FROM Books_Author NATURAL JOIN Shopping_Cart
WHERE CustomerID IN
    (SELECT Shopping_Cart.CustomerID
     FROM Shopping_Cart
     JOIN Customer ON Shopping_Cart.CustomerID = Customer.CustomerID
     GROUP BY Shopping_Cart.CustomerID
     HAVING SUM(Cost) >
        (SELECT AVG(TotalSpent)
         FROM (SELECT SUM(Cost) AS TotalSpent
              FROM Shopping_Cart
              GROUP BY CustomerID)))
);
```

Appendix D: Project Checkpoint 4

1. Provide a current version of your ER Diagram and Relational Model as per Project Checkpoint 03. **If you were instructed to change the model for Project Checkpoint 03, make sure you use the revised versions of your models.**

ER Diagram:



Relational Model:

ADMIN(AdminID#, FirstName, LastName, Email)

INVENTORY_ORDER(Order_No#, Order_Date, InventoryID^(FK), AdminID^(FK), BookID^(FK), Quantity, Cost)

★ **ADMIN_ORDERS**(Order_No#^(FK), AdminID^(FK))

BOOK(BookID#, PublisherID^(FK), InventoryID^(FK), Title, Year, Genre, ISBN, Price)

ORDERED_BY(Order_No#, BookID^(FK))

PUBLISHERS(PublisherID#, PName)

SUPPLIES(Order_No^(FK), PublisherID^(FK))

BOOKS_AUTHOR(BookID#^(FK), Author_Name)

INVENTORY(InventoryID#, Quantity)

CUSTOMER(CustomerID#, FirstName, LastName, Email, Phone_Number)

SHOPPING_CART(Order_No#, Order_Date, BookID^(FK), Quantity, Cost)

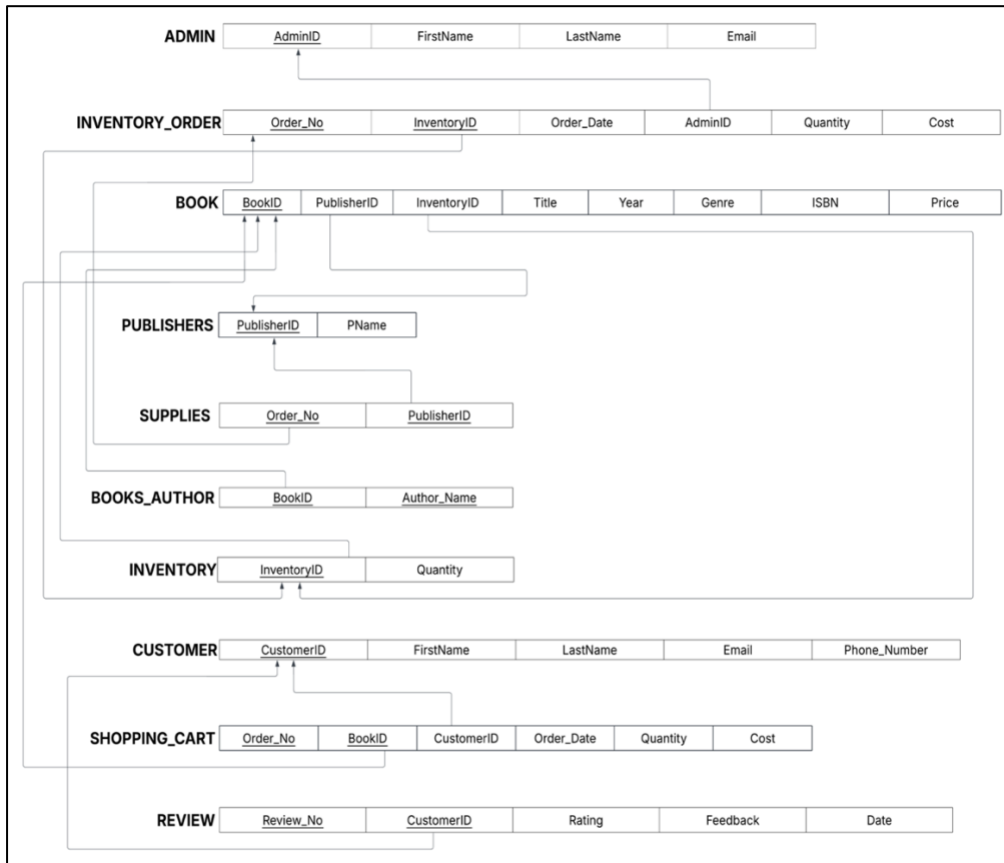
★ **CUSTOMER_ORDERS**(Order_No^(FK), CustomerID^(FK))

REVIEW(Review_No#, CustomerID^(FK), Rating, Feedback, Date)

CUSTOMER_BOOKS(CustomerID#^(FK), BookID#^(FK), Title, ISBN)

★ New Relation added after examining Functional Dependencies in old schema

Old Schema:



2. For each relation schema in your model, indicate the functional dependencies. Think carefully about what you are modeling here - make sure you consider all the possible dependencies in each relation and not just the ones from your primary keys. For example, a customer's credit card number is unique, and so will uniquely identify a customer even if you have another key in the same table (in fact, if the customer can have multiple credit card numbers, the dependencies can get even more involved).

- **Admin Relation: (Multiple people cannot have the same Email, so it is a candidate key)**

{AdminID} → {FirstName, LastName, Email}

{Email} → {AdminID, FirstName, LastName}

- **Inventory_Order Relation:**

{Order_No, InventoryID} → {Order_Date, AdminID, Quantity, Cost}

{Order_No} → {AdminID}

- **Book Relation:**

{BookID} → {PublisherID, InventoryID, Title, Year, Genre, ISBN, Price}

{InventoryID} → {BookID, PublisherID, Title, Year, Genre, ISBN, Price}

{ISBN} → {BookID, InventoryID, PublisherID, Title, Year, Genre, Price}

- **Publishers Relation:**

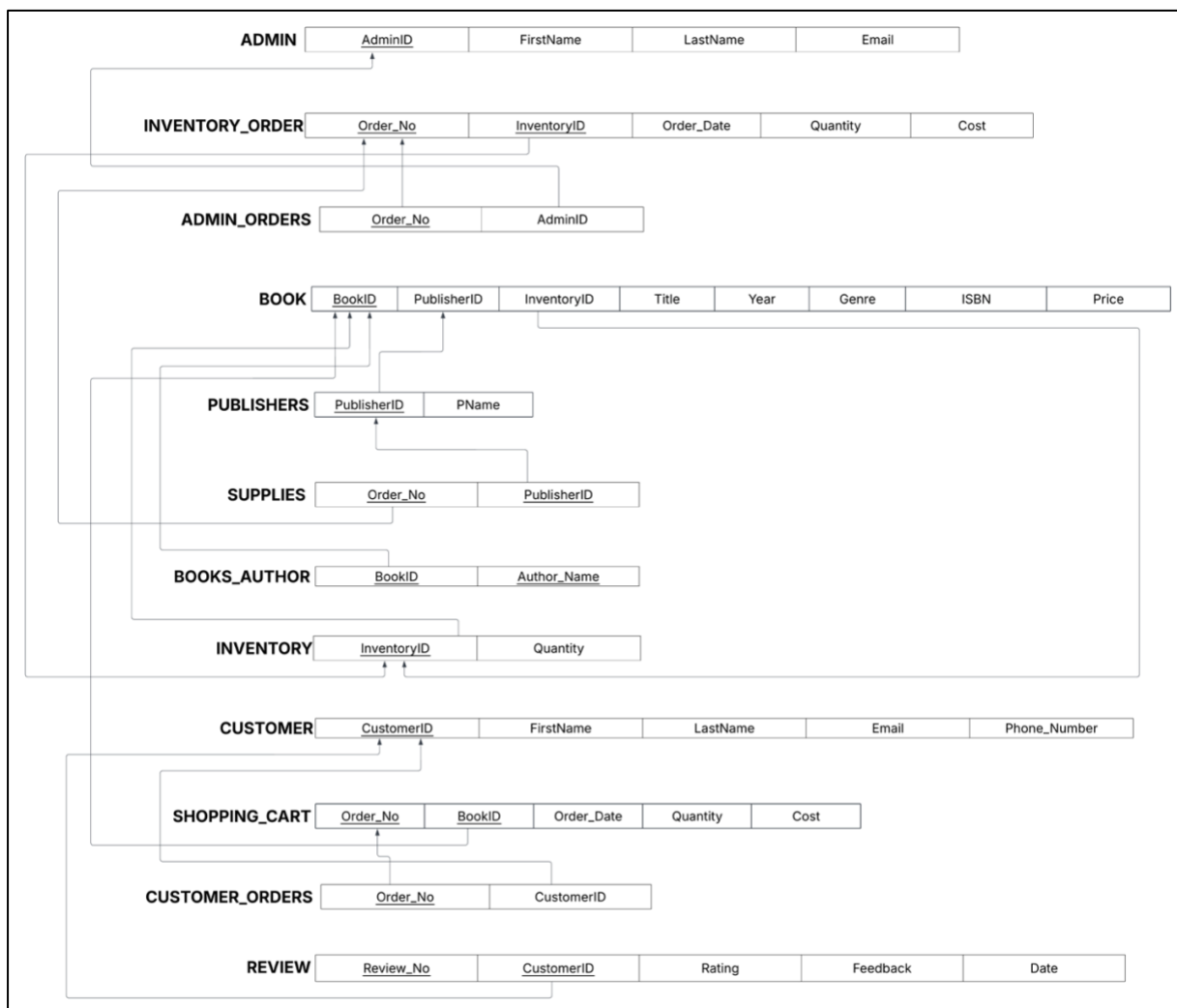
{PublisherID} → {PName}

- **Books Author Relation:**

{BookID} → {Author_Name}

- **Customer Relation: (Multiple cannot have the same Email or Phone Number, so they are candidate keys)**
 {CustomerID} -> {FirstName, LastName, Email, Phone_Number}
 {Email} -> {CustomerID, FirstName, LastName, Phone_Number}
 {Phone_Number} -> {CustomerID, FirstName, LastName, Email}
- **Review Relation:**
 {Review_No, CustomerID} -> {Rating, Feedback, Date}
- **Shopping Cart Relation:**
 {Order_No, BookID} -> {CustomerID, Order_Date, Quantity, Cost}
 {Order_No} -> {CustomerID}
- **Inventory Relation:**
 {InventoryID} -> {Quantity}
- **Supplies Relation:**
 {Order_No} -> {PublisherID}

New Schema:



3. For each relation schema in your model, determine the highest normal form of the relation. If the relation is not in 3NF, rewrite your relation schema so that it is in at least 3NF.

Apart from the Shopping_Cart and Inventory_Order relations, all relations are in 3NF.

1. For the Shopping_Cart relation there is a partial dependency of CustomerID on the Order_No. A separate relation is needed for Order_No and CustomerID. An additional relation **CUSTOMER_ORDERS**(Order_No^(FK), CustomerID^(FK)) was added to the schema
 2. For the Inventory_Order relation there is a partial dependency of AdminID on the Order_No. A separate relation is needed for Order_No and AdminID. An additional relation **ADMIN_ORDERS**(Order_No^(FK), AdminID^(FK)) was added to the schema
4. For each relation schema in your model that is in 3NF but not in BCNF, either rewrite the relation schema to BCNF or provide a short justification for why this relation should be an exception to the rule of putting relations into BCNF.
Each relational schema in our model is in BCNF.

All relations are in BCNF.

5. For your database, propose at least two interesting views that can be built from your relations. These views must involve joining at least two tables together each and must include some kind of aggregation in the view. Each view must also be able to be described by a one or two sentence description in plain English. Provide the code for constructing your views along with the English language description of what the view is supposed to be providing.

View 1: Customer Orders - This view records the total amount spent by each customer in the bookstore. Provides information about frequent customers.

```
CREATE VIEW Customer_Orders_Total AS
SELECT CUSTOMER.CustomerID, CUSTOMER.FirstName, CUSTOMER.LastName,
CUSTOMER.Email, COUNT(SHOPPING_CART.Order_No) AS Total_Orders ,
SUM(SHOPPING_CART.Cost) AS TotalSpent
FROM CUSTOMER_ORDERS
JOIN SHOPPING_CART ON CUSTOMER_ORDERS.Order_No = SHOPPING_CART.Order_No
JOIN CUSTOMER ON CUSTOMER.CustomerID = CUSTOMER_ORDERS.CustomerID
GROUP BY CUSTOMER_ORDERS.CustomerID
```

View 2: Customer Reviews - This view displays the average rating per customer and total number of ratings they have posted. Includes customer information. This provides information about customer satisfaction and a contact point for customers that have not had a good experience.

```
CREATE VIEW Customer_Reviews AS
SELECT CUSTOMER.FirstName, CUSTOMER.LastName, CUSTOMER.Email,
COUNT(REVIEW.Review_No), AVG(REVIEW.Rating)
FROM CUSTOMER
JOIN REVIEW ON CUSTOMER.CustomerID = REVIEW.CustomerID
GROUP BY CUSTOMER.CustomerID;
```